

Deadlock Team Comp Analyzer - line-by-line deep dive

Dit is de extra uitgebreide versie van de projectuitleg. De vorige deep dive blijft bestaan. Dit document gaat veel dichter op de code zitten: per bestand worden de belangrijkste regels en codeblokken besproken, met extra uitleg over waarom ze bestaan en hoe ze samen de pipeline vormen.

De rode draad:

1. Configuratie en paden
2. Deadlock matches ophalen
3. Ruwe API-response normaliseren
4. Account IDs verzamelen
5. Statlocker ppScores ophalen
6. Dataset bouwen
7. Neural network trainen
8. Experimenten vergelijken
9. Streamlit-app gebruiken voor inference

Belangrijke gedachte: bijna elk bestand heeft een duidelijke rol. ``src/`` bevat herbruikbare code. ``scripts/`` bevat uitvoerbare pipeline-stappen. ``app/`` bevat de gebruikersinterface. ``data/`` en ``models/`` zijn outputmappen.

1. Root-bestanden

1.1 ``.env``

``.env`` staat lokaal op de VM en hoort niet in GitHub. Dit bestand bevat secrets, vooral:

```
STATLOCKER_API_KEY=...
```

Die API key wordt gebruikt door ``src/statlocker_api.py``. Als deze key in GitHub terechtkomt, kan iemand anders jouw quota gebruiken. Daarom staat ``.env`` in ``.gitignore``.

1.2 ``.env.example``

``.env.example`` is een voorbeeldbestand. Hierin staat geen echte key, maar wel welke environment variable nodig is.

Goed patroon:

```
STATLOCKER_API_KEY=your_key_here
```

Zo kan iemand anders de repo clonen en begrijpen wat hij zelf lokaal moet invullen.

1.3 `.gitignore`

De `.gitignore` hoort minstens deze dingen te negeren:

```
.env
.venv/
  pycache /
*.pyc
data/*.jsonl
data/*.csv
data/pp_scores.json
data/*.log
```

Waarom datasets meestal genegeerd worden:

- ze zijn groot;
- ze kunnen opnieuw gegenereerd worden;
- ze veranderen vaak;
- soms bevatten ze indirect gevoelige accountinformatie.

Kleine documentatiebestanden, scripts en modelmetadata mogen wel in GitHub.

1.4 `requirements.txt`

Dit bestand beschrijft de Python dependencies. Voor dit project zijn vooral belangrijk:

```
torch
streamlit
numpy
```

Tijdens het examen moet je kunnen uitleggen: als de VM opnieuw moet installeren, gebruik je:

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt
```

2. `scripts/common.py`

Dit is een klein maar centraal bestand. Veel scripts importeren hieruit dezelfde paden en helperfuncties.

2.1 Imports

```
from __future__ import annotations
```

Deze regel zorgt dat type hints pas later geevalueerd worden. Dat maakt moderne type syntax makkelijker en voorkomt sommige importproblemen.

```
import json
import sys
from pathlib import Path
from typing import Any
```

Regel per regel:

- `json`: nodig om JSON en JSONL te lezen/schrijven.
- `sys`: nodig om `src/` aan het importpad toe te voegen.
- `Path`: moderne manier om bestandspaden te behandelen.
- `Any`: type hint voor willekeurige JSON-data.

2.2 Repo-root bepalen

```
REPO_ROOT = Path(__file__).resolve().parents[1]
```

`\_\_file\_\_` is het pad naar `scripts/common.py`.

`resolve()` maakt daar een absoluut pad van.

`parents[1]` gaat twee niveaus omhoog:

```
/root/DL-team-comp-analyzer/scripts/common.py
parents[0] = /root/DL-team-comp-analyzer/scripts
parents[1] = /root/DL-team-comp-analyzer
```

Dus `REPO\_ROOT` is de projectmap.

2.3 `src/` importeerbaar maken

```
SRC_DIR = REPO_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
```

Omdat `scripts/` en `src/` naast elkaar staan, kan Python `src/deadlock\_api.py` niet vanzelf importeren wanneer je een script uitvoert vanuit `scripts/`. Deze regels lossen dat op.

Zonder dit zou dit falen:

```
from deadlock_api import DeadlockApiClient
```

Met `SRC\_DIR` in `sys.path` werkt dat wel.

2.4 Centrale datapaden

```
DATA_DIR = REPO_ROOT / "data"
CURRENT_PATCH = "2026-05-22"
MATCHES_PATH = DATA_DIR / f"matches {CURRENT_PATCH}.jsonl"
ACCOUNTS_PATH = DATA_DIR / f"accounts_{CURRENT_PATCH}.txt"
```



```
PP SCORES PATH = DATA DIR / "pp scores.json"
DATASET PATH = DATA DIR / f"team comp dataset {CURRENT_PATCH}.csv"
FETCH_STATE_PATH = DATA_DIR / f"fetch_state_{CURRENT_PATCH}.json"
```

Hier definieert het project zijn standaardbestanden.

Als `CURRENT\_PATCH = "2026-05-22"`, dan worden de defaults:

```
data/matches 2026-05-22.jsonl
data/accounts 2026-05-22.txt
data/pp scores.json
data/team comp dataset 2026-05-22.csv
data/fetch_state_2026-05-22.json
```

Let op: voor je 20k dataset gebruik je soms expliciete paden zoals:

```
data/team_comp_dataset_2026-05-22_20k.csv
```

Dat doe je via command line arguments. De defaults zijn vooral gemak.

2.5 Parent folder garanderen

```
def ensure_parent(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
```

Als je schrijft naar `data/something.json`, moet `data/` bestaan. Deze functie maakt de parent folder aan indien nodig.

- `parents=True`: maak ook ontbrekende tussenmappen.
- `exist\_ok=True`: geef geen error als de map al bestaat.

2.6 JSONL lezen

```
def read_jsonl(path: Path) -> list[dict[str, Any]]:
    if not path.exists():
        return []
```

Als het bestand nog niet bestaat, krijg je een lege lijst. Daardoor kunnen scripts veilig starten op een nieuwe dataset.

```
rows: list[dict[str, Any]] = []
for line in path.read_text(encoding="utf-8").splitlines():
    if not line.strip():
        continue
    payload = json.loads(line)
    if isinstance(payload, dict):
        rows.append(payload)
return rows
```

Per regel:

- Lees het bestand als tekst.
- Splits op regels.
- Sla lege regels over.
- Parse elke regel als JSON.
- Alleen dicts worden toegevoegd.

Waarom JSONL? Je kan matches append'en zonder een grote JSON-array telkens opnieuw te schrijven.

2.7 JSONL append'en

```
def append_jsonl(path: Path, rows: list[dict[str, Any]]) -> None:
    ensure_parent(path)
    with path.open("a", encoding="utf-8") as file:
        for row in rows:
            file.write(json.dumps(row) + "\n")
```

Belangrijk:

- `"a"` betekent append mode.
- Elke match komt op een nieuwe regel.
- `ensure\_parent` voorkomt dat schrijven faalt omdat `data/` ontbreekt.

2.8 JSON cache laden

```
def load_json(path: Path, default: Any) -> Any:
    if not path.exists():
        return default
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return default
```

Dit wordt gebruikt voor state files en `pp\_scores.json`.

Als het bestand ontbreekt of corrupt is, krijg je `default`. Dat maakt de scripts robuust, maar betekent ook: als een JSON kapot is, start je stil met default. Bij ppScores moet je dus opletten dat je cache niet per ongeluk corrupt is.

2.9 IDs lezen en schrijven

```
def read_ids(path: Path) -> list[str]:
    if not path.exists():
        return []
    return [line.strip() for line in path.read_text(encoding="utf-8").splitlines() if line.strip()]
```

Accountbestanden zijn simpele tekstbestanden:

123

456
789

Elke regel is één account ID.

```
def write_ids(path: Path, ids: list[str]) -> None:
    ensure parent(path)
    path.write_text("\n".join(ids) + ("\n" if ids else ""), encoding="utf-8")
```

Deze functie schrijft één ID per regel.

2.10 Batches maken

```
def chunked[T](values: list[T], size: int) -> list[list[T]]:
    return [values[index : index + size] for index in range(0, len(values), size)]
```

Voorbeeld:

```
values = [1,2,3,4,5]
size = 2
result = [[1,2], [3,4], [5]]
```

Dit is belangrijk voor Statlocker, omdat je accounts in batches van maximaal 100 ophaalt.

3. `src/env\_utils.py`

Dit bestand laadt `.env` zonder extra dependency zoals `python-dotenv`.

3.1 Imports

```
import os
from pathlib import Path
```

- `os.environ` bevat environment variables.
- `Path` gebruikt het bestandspad naar `.env`.

3.2 Repo-root en `.env` vinden

```
repo root = Path( file ).resolve().parents[1]
env path = repo root / ".env"
if not env path.exists():
    return
```

Omdat `env\_utils.py` in `src/` staat, is `parents[1]` opnieuw de repo-root.

Als `.env` niet bestaat, doet de functie niets. Dat is goed: voor Deadlock API heb je geen key nodig.

3.3 Regels parsen

```
for raw line in env.path.read_text(encoding="utf-8").splitlines():
    line = raw line.strip()
    if not line or line.startswith("#") or "=" not in line:
        continue
```

Hier worden lege regels, comments en ongeldige regels overgeslagen.

```
key, value = line.split("=", 1)
key = key.strip()
value = value.strip()
```

`split("=", 1)` splitst maar op de eerste `=`. Daardoor kan een value zelf eventueel nog een `=` bevatten.

3.4 Bestaande env vars niet overschrijven

```
if not key or key in os.environ:
    continue
```

Als je in de shell al een environment variable gezet hebt, wint die boven `.env`. Dat is een nette conventie.

3.5 Quotes verwijderen

```
if len(value) >= 2 and value[0] == value[-1] and value[0] in {'"', "'"}:
    value = value[1:-1]
```

Dit ondersteunt:

```
STATLOCKER_API_KEY="abc"
STATLOCKER_API_KEY='abc'
```

3.6 In environment zetten

```
os.environ[key] = value
```

Vanaf dit moment kan andere code doen:

```
os.getenv("STATLOCKER_API_KEY")
```

4. `src/deadlock\_api.py`

Dit is de HTTP-client voor de Deadlock API.

4.1 Imports

```
import json
import math
from typing import Any
from urllib.parse import urlencode
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen
```

Regel per regel:

- `json`: API-response parsen.
- `math`: retry-after schatten bij quota-info.
- `Any`: response kan dict, list, etc. zijn.
- `urlencode`: queryparameters correct omzetten naar URL.
- `HTTPError`, `URLError`: netwerkfouten onderscheiden.
- `Request`, `urlopen`: standard library HTTP.

4.2 Base URL

```
DEFAULT_GAME_API_BASE = "https://api.deadlock-api.com/v1"
```

Alle endpoints worden relatief aan deze base URL gebouwd.

4.3 Custom exceptions

```
class DeadlockApiError(RuntimeError):
    """Raised when the Deadlock API request fails."""
```

Eigen exceptionklassen maken de rest van de code duidelijker. `fetch\_matches.py` kan specifiek `DeadlockRateLimitError` afhandelen.

```
def init (self, message: str, *, status code: int | None = None, retry after seconds: float | None = None) -> None:
    super(). init (message)
    self.status code = status code
    self.retry_after_seconds = retry_after_seconds
```

Naast de foutboodschap bewaart de exception ook:

- HTTP statuscode;
- aanbevolen wachttijd.

4.4 Client initialisatie

```
class DeadlockApiClient:
    def init (self, game api base: str = DEFAULT_GAME_API_BASE, timeout seconds: int = 20) -> None:
        self.game_api_base = game_api_base.rstrip("/")
```



```
self.timeout_seconds = timeout_seconds
```

`rstrip("/")` voorkomt dubbele slashes:

```
https://api.../v1//matches/metadata
```

4.5 Bulk metadata endpoint

```
def fetch_bulk_match_metadata(self, **query_params: Any) -> Any:
    query = encode_query_params(query_params)
    url = f"{self.game_api_base}/matches/metadata"
    if query:
        url = f"{url}?{query}"
    return self._get_json(url)
```

Dit bouwt bijvoorbeeld:

```
https://api.deadlock-api.com/v1/matches/metadata?limit=100&game_mode=normal
```

4.6 Request headers

```
request = Request(
    url,
    headers={
        "Accept": "application/json",
        "User-Agent": "dl-team-comp-analyzer/0.1",
    },
)
```

`Accept: application/json` zegt dat we JSON verwachten.

`User-Agent` is netjes voor API logging.

4.7 Response lezen

```
with urlopen(request, timeout=self.timeout_seconds) as response:
    body = response.read().decode("utf-8")
```

`timeout\_seconds` voorkomt dat het script oneindig blijft hangen.

4.8 Rate limits

```
except HTTPError as exc:
    retry_after_seconds = extract_retry_after_seconds(exc, response.body)
    if exc.code == 429:
        raise DeadlockRateLimitError(...)
```

HTTP 429 betekent rate limit. In plaats van te crashen met een onduidelijke

traceback, geeft de code een duidelijke exception aan `fetch\_matches.py`.

4.9 JSON parsing

```
try:
    payload = json.loads(body)
except json.JSONDecodeError as exc:
    raise DeadlockApiError(...)
```

Als de API HTML of kapotte JSON terugstuurt, wordt dat een nette fout.

4.10 Queryparameters encoden

```
def encode_query_params(query_params: dict[str, Any]) -> str:
    cleaned: list[tuple[str, str]] = []
```

De functie bouwt alleen geldige queryparameters.

```
if value is None:
    continue
```

`None` wordt niet in de URL gezet.

```
if isinstance(value, bool):
    cleaned.append((key, "true" if value else "false"))
```

Booleans worden API-vriendelijk gemaakt.

```
if isinstance(value, (list, tuple, set)):
    cleaned.append((key, ",".join(str(item) for item in value)))
```

Lijsten worden comma-separated.

```
return urlencode(cleaned)
```

`urlencode` zorgt dat speciale tekens correct escaped worden.

5. `src/bulk\_extract.py`

Dit bestand is defensief geschreven omdat API-responses soms anders genest kunnen zijn.

5.1 Hoofdfunctie

```
def extract_match_payloads(payload: Any) -> list[dict[str, Any]]:
```

Input kan een list, dict of iets anders zijn. Output is altijd een lijst met matchdicts.

5.2 Als payload een lijst is

```
if isinstance(payload, list):
    direct = [item for item in payload if looks like match(item)]
    if direct:
        return direct
```

Als de API direct een lijst matches teruggeeft, worden die meteen gebruikt.

```
nested: list[dict[str, Any]] = []
for item in payload:
    nested.extend(extract match payloads(item))
return nested
```

Als de items zelf wrappers zijn, zoekt de functie recursief verder.

5.3 Als payload een dict is

```
for key in ("matches", "results", "items", "data"):
    candidate = payload.get(key)
```

Veel APIs stoppen lijsten in keys zoals `data` of `results`. De functie ondersteunt meerdere vormen.

```
if looks like match(payload):
    return [payload]
```

Als de dict zelf een match is, wordt die als single-item lijst teruggegeven.

5.4 Match herkennen

```
def looks like match(candidate: Any) -> bool:
    if not isinstance(candidate, dict):
        return False
```

Alleen dicts kunnen matchpayloads zijn.

```
if "match info" in candidate and isinstance(candidate["match info"], dict):
    return True
```

Sommige responses stoppen alles onder `match\_info`.

```
if {"match id", "players"} <= set(candidate.keys()):
    return True
```

Een match met `match\_id` en `players` is duidelijk bruikbaar.

```
return (
```



```

    "players" in candidate
    and isinstance(candidate["players"], list)
    and pick first(candidate, "winning team", "winner") is not None
)

```

Als er players en een winner zijn, lijkt het ook op een match.

6. `src/patch\_history.py`

Dit bestand bepaalt bij welke patch een match hoort.

6.1 Dataclass

```

@dataclass(frozen=True)
class PatchRelease:
    patch name: str
    effective from utc: datetime
    source: str

```

`frozen=True` betekent immutable: een patchrelease wordt na aanmaak niet meer aangepast.

Velden:

- `patch\_name`: bijvoorbeeld `2026-05-22`.
- `effective\_from\_utc`: exacte starttijd van de patch.
- `source`: waar die info vandaan komt.

6.2 Patchlijst

```

PATCH RELEASES: tuple[PatchRelease, ...] = (
    PatchRelease(
        patch name="2026-05-22",
        effective from utc=datetime(2026, 5, 22, 21, 52, 5, tzinfo=UTC),
        source="official Gameplay Update - 05-22-2026 release time",
    ),
    ...
)

```

De nieuwste patch staat bovenaan. Dat is belangrijk voor de lookup.

6.3 Patch infereren

```

def infer patch from start time(start time s: int | None) -> tuple[str, str]:
    if start time s is None:
        return "Unknown", "missing start_time"

```

Zonder starttijd kan de code geen patch bepalen.

```
match_time = datetime.fromtimestamp(start_time_s, tz=UTC)
```

UNIX timestamp wordt een UTC datetime.

```
for patch in PATCH_RELEASES:
    if match_time >= patch.effective_from_utc:
        selected = patch
        break
```

Omdat nieuwste patches bovenaan staan, is de eerste patch waarvan de matchtijd later is de juiste.

Voorbeeld:

```
match_time = 2026-05-23
eerste_patch <= match_time is 2026-05-22
```

7. `src/match\_parser.py`

Dit is een van de belangrijkste files. Het maakt van rommelige API-data een vaste matchrepresentatie.

7.1 Type alias

```
HeroResolver = Callable[[int | str | None], str]
```

Een `HeroResolver` is een functie die een hero ID omzet naar een naam. Als er geen resolver gegeven is, gebruikt de parser:

```
lambda hero_id: f"Hero {hero_id}" if hero_id is not None else "Unknown Hero"
```

7.2 Mogelijke keys

```
PLAYER_LIST_KEYS = (
    "players",
    "match_players",
    "player_slots",
    "participants",
    "members",
)
```

API's zijn niet altijd consistent. Daarom zoekt de parser naar meerdere mogelijke keys voor dezelfde betekenis.

Hetzelfde gebeurt voor patch en winner:

```
PATCH_KEYS = ("patch", "patch name", "version", "client version", "build id")
_WINNER_KEYS = ("winner", "winner_team", "winning_team", "winning_side", "victorious_
```

```
team")
```

7.3 `PlayerView`

```
@dataclass
class PlayerView:
    player name: str
    account id: str
    hero id: str
    hero name: str
    pp score: str
    team key: str
    player_slot: int | None
```

Elke speler in een match wordt genormaliseerd naar deze velden.

Belangrijk:

- `account\_id` is nodig voor Statlocker.
- `hero\_id` is nodig voor het model.
- `team\_key` bepaalt team 1/team 2.
- `player\_slot` helpt sorteren in vaste volgorde.

7.4 `MatchView`

```
@dataclass
class MatchView:
    match id: str
    start time s: int | None
    start time utc: str
    patch: str
    patch source: str
    winner: str
    winner team index: int | None
    team 1 label: str
    team 2 label: str
    team 1 average badge: str
    team 2 average badge: str
    team 1 players: list[PlayerView]
    team_2_players: list[PlayerView]
```

Dit is het centrale object na parsing.

`winner\_team\_index` is uiteindelijk het label voor training:

```
0 -> team 1 wint
1 -> team 2 wint
```

7.5 `build\_match\_view`


```
def build_match_view(metadata: dict[str, Any], hero_resolver: HeroResolver | None = None) -> MatchView:
```

Input is een ruwe matchdict. Output is een `MatchView`.

```
root = unwrap_match_payload(metadata)
players = find_players(root)
if not players:
    raise ValueError(...)
```

De parser unwrapt eerst de payload en zoekt dan spelers. Zonder spelers is de match onbruikbaar.

7.6 Spelers groeperen

```
grouped: dict[str, list[PlayerView]] = {}
ordered_team_keys: list[str] = []
```

`grouped` wordt bijvoorbeeld:

```
{
    "team 1": [PlayerView(...), ...],
    "team 2": [PlayerView(...), ...],
}
```

7.7 Loop over spelers

```
for index, raw_player in enumerate(players, start=1):
    team_key = extract_team_key(raw_player, fallback=f"team {1 if index <= len(players) / 2 else 2}")
```

Als de API geen team key bevat, gebruikt de parser een fallback: eerste helft spelers team 1, tweede helft team 2.

```
if team_key not in grouped:
    grouped[team_key] = []
    ordered_team_keys.append(team_key)
```

Nieuwe teams worden toegevoegd.

```
hero_id = _extract_hero_id(raw_player)
```

De hero ID wordt uit verschillende mogelijke API-velden gehaald.

7.8 PlayerView bouwen

```
grouped[team_key].append(
    PlayerView(
```

```

        player name= extract player name(raw player, index=index),
        account id= extract account id(raw player, index=index),
        hero id="" if hero id is None else str(hero id),
        hero name=hero lookup(hero id),
        pp score= extract pp score(raw player),
        team key=team key,
        player slot= extract player slot(raw player),
    )
)

```

Dit is de normalisatie:

- onbekende hero wordt lege string;
- ID's worden strings;
- naam en account krijgen fallback als nodig;
- ppScore uit Deadlock zelf is meestal niet bruikbaar, maar veld blijft bestaan.

7.9 Team 1 en team 2 kiezen

```

if "team 1" in grouped:
    team 1 key = "team 1"
else:
    team_1_key = ordered_team_keys[0] if ordered_team_keys else "team_1"

```

Als de API expliciet `team\_1` heeft, gebruiken we dat. Anders nemen we het eerste gevonden team.

Team 2:

```

if "team 2" in grouped:
    team 2 key = "team 2"
else:
    fallback keys = [key for key in ordered team keys if key != team 1 key]
    team_2_key = fallback_keys[0] if fallback_keys else "team_2"

```

7.10 Spelers sorteren

```

grouped[team 1 key] = sorted players(grouped.get(team 1 key, []))
grouped[team_2_key] = _sorted_players(grouped.get(team_2_key, []))

```

Sorting is belangrijk: je wil consistente slots `hero\_1` tot `hero\_6`. Het model zelf gebruikt pooling, dus de volgorde is minder cruciaal dan bij een gewone MLP, maar consistente data blijft beter.

7.11 Winner en patch

```

winner = extract winner(root)
winner label = format winner(winner, team 1 key, team 2 key)
winner_team_index = _normalize_winner_index(winner)

```

```
start time s = extract start time(root)
patch, patch_source = _resolve_patch(root, start_time_s)
```

Hier worden de labels gemaakt.

Het trainingsdoel komt uit `winner\_team\_index`.

De patch komt uit de API als die beschikbaar is, of uit `patch\_history.py` via starttijd.

7.12 Naar dict

```
def match view to dict(match view: MatchView) -> dict[str, Any]:
    return {
        "match id": match view.match id,
        ...
        "team 1 players": [asdict(player) for player in match view.team 1 players],
        "team 2 players": [asdict(player) for player in match view.team 2 players],
    }
```

Dat is wat uiteindelijk in JSONL komt. Dat bestand is de bron voor latere stappen.

8. `scripts/fetch\_matches.py`

Dit is de eerste echte pipeline-stap.

8.1 Imports

```
from common import CURRENT PATCH, FETCH STATE PATH, MATCHES PATH, append jsonl, load
json, read_jsonl, write_json
```

Dit haalt defaults en helperfuncties uit `common.py`.

```
from bulk extract import extract match payloads
from deadlock api import DeadlockApiClient, DeadlockApiError, DeadlockRateLimitError
from match_parser import build_match_view, match_view_to_dict
```

Hier zie je de samenwerking:

- API client haalt data.
- bulk extractor vindt matches in de response.
- parser normaliseert matches.

8.2 CLI arguments

```
parser.add argument("--target-count", type=int, default=10000)
parser.add argument("--batch-size", type=int, default=100)
parser.add argument("--timeout-seconds", type=int, default=45)
parser.add_argument("--sleep-seconds", type=float, default=0.35)
```


Deze bepalen hoeveel matches je wil en hoe snel je requests doet.

```
parser.add_argument("--resume", action="store_true")
```

Met `--resume` gebruikt het script de state-file om verder te gaan.

```
parser.add_argument("--required-patch", default=CURRENT_PATCH)
```

Alleen matches van deze patch worden opgeslagen.

```
parser.add_argument(
    "--game-mode",
    choices=("normal", "street brawl", "explore n y c", "internal"),
    default="normal",
)
```

Dit voorkomt dat Street Brawl in je dataset komt, zolang je `normal` gebruikt.

8.3 Startstatus

```
client = DeadlockApiClient(timeout seconds=args.timeout seconds)
existing_match_ids = {str(match.get("match id")) for match in read_jsonl(args.output)
}
state = load_json(args.state_file, {}) if args.resume else {}
```

Betekenis:

- Maak API client.
- Lees bestaande output, zodat duplicaten niet worden toegevoegd.
- Lees state alleen als `--resume` meegegeven is.

8.4 Match ID cursor

```
current_min_match_id = state.get("next min match id", args.min_match_id)
current_max_match_id = state.get("next_max_match_id", args.max_match_id)
```

Bij `order\_direction desc` schuift `current\_max\_match\_id` steeds omlaag. Zo ga je terug in de tijd.

8.5 Hoofdloop

```
while len(existing_match_ids) < args.target_count:
```

De loop gaat door tot het outputbestand genoeg unieke matches bevat.

Let op: als je al 8900 matches hebt en target is 10000, haalt het script nog maar ongeveer 1100 nieuwe matches op.

8.6 Query bouwen


```

query = {
    "limit": args.batch size,
    "order by": "match id",
    "order direction": args.order direction,
    "min match id": current min match id,
    "max match id": current max match id,
    "include info": True,
    "include player info": True,
    ...
    "game mode": args.game mode,
}

```

Belangrijkste keuzes:

- `include\_player\_info=True`: nodig voor heroes en accounts.
- player stats/items/objectives staan uit om response kleiner te houden.
- `game\_mode=normal`: juiste spelmodus.

8.7 API call met retrygedrag

```

try:
    payload = client.fetch bulk match metadata(**query)
except DeadlockRateLimitError as exc:
    wait seconds = exc.retry after seconds or args.rate limit sleep seconds
    time.sleep(wait seconds)
    continue

```

Bij rate limit wacht het script en probeert het opnieuw.

```

except DeadlockApiError as exc:
    consecutive errors += 1
    if consecutive errors >= args.max consecutive errors:
        return 1

```

Bij herhaalde fouten stopt het script netjes.

8.8 Matches extraheren en normaliseren

```

raw_matches = extract_match_payloads(payload)

```

Dit maakt van de API-response een lijst ruwe matchdicts.

```

summaries, skipped other patch = normalize_matches(...)
append_jsonl(args.output, summaries)

```

Alleen bruikbare, juiste-patch matches worden opgeslagen.

8.9 Cursor updaten

```

if args.order direction == "desc":
    current_max_match_id = min(match_ids) - 1
else:
    current_min_match_id = max(match_ids) + 1

```

Als je descending werkt, was de laagste ID in de batch het verst terug. De volgende batch moet daaronder verdergaan.

8.10 `normalize\_matches`

```

for raw match in raw matches:
    try:
        match view = build match view(raw match)
    except ValueError:
        continue

```

Kapotte matches worden overgeslagen.

```

if match view.match_id in existing match_ids:
    continue

```

Duplicaten worden overgeslagen.

```

if not allow missing start time and match view.start time s is None:
    continue

```

Zonder starttijd kan patchfiltering onbetrouwbaar zijn.

```

if required patch and match view.patch != required patch:
    skipped other patch += 1
    continue

```

Dit is de patchfilter.

```

summaries.append(match view to dict(match view))
existing_match_ids.add(match_view.match_id)

```

De match wordt opgeslagen en onmiddellijk in de dedupe-set gezet.

9. `scripts/extract\_accounts.py`

Dit script haalt unieke account IDs uit de match JSONL.

9.1 Arguments

```

parser.add_argument("--matches", type=Path, default=MATCHES_PATH)
parser.add_argument("--output", type=Path, default=ACCOUNTS_PATH)

```

Input is het matchbestand. Output is een tekstbestand met account IDs.

9.2 Main

```
accounts = sorted(collect accounts(read jsonl(args.matches)), key=int)
write_ids(args.output, accounts)
```

Stap voor stap:

1. Lees JSONL.
2. Verzamel accounts.
3. Sorteer numeriek.
4. Schrijf één ID per regel.

9.3 Accountcollectie

```
for match in matches:
    for team key in ("team 1 players", "team 2 players"):
        for player in match.get(team key, []) or []:
            account id = str(player.get("account id", "")).strip()
            if account id and not account id.startswith("unknown-"):
                accounts.add(account_id)
```

Belangrijk:

- Beide teams worden gelezen.
- Lege IDs worden genegeerd.
- Fallback IDs zoals `unknown-...` worden genegeerd.
- `set` voorkomt duplicaten.

10. `src/statlocker\_api.py`

Dit is de client voor Statlocker.

10.1 Base URL

```
DEFAULT_STATLOCKER_API_BASE = "https://statlocker.gg/api"
```

De endpoint calls worden hierop gebouwd.

10.2 API key laden

```
load repo env()
self.api key = api key or os.getenv("STATLOCKER API KEY")
if not self.api key:
    raise StatlockerApiError(...)
```

Volgorde:

1. Lees `.env`.

2. Kijk of `api\_key` expliciet is meegegeven.
3. Anders gebruik `STATLOCKER\_API\_KEY`.
4. Als die ontbreekt, stop met duidelijke fout.

10.3 Single profile en batch profile

```
def fetch_profile(self, account id: int | str) -> Any:
    return self._get_json(f"{self.api_base}/public/profile/{account_id}")
```

Voor één account.

```
def fetch_batch_profiles(self, account ids: list[int | str]) -> Any:
    return self._post_json(f"{self.api_base}/public/profiles", account_ids)
```

Voor meerdere accounts tegelijk. Dit is wat `fetch\_pp\_scores.py` gebruikt.

10.4 Headers

```
def headers(self) -> dict[str, str]:
    return {
        "Accept": "application/json",
        "User-Agent": "dl-team-comp-analyzer/0.1",
        "X-API-Key": self.api key,
    }
```

Belangrijk: Statlocker gebruikt `X-API-Key`, niet `Authorization: Bearer`.

10.5 POST body

```
body = json.dumps(payload).encode("utf-8")
headers = self.headers()
headers["Content-Type"] = "application/json"
request = Request(url, headers=headers, data=body, method="POST")
```

Voor batch profiles stuur je de account IDs als JSON-array.

10.6 Error handling

```
error_cls = StatlockerRateLimitError if exc.code == 429 else StatlockerApiError
raise error_cls(...)
```

Bij 429 kan `fetch\_pp\_scores.py` langer wachten. Andere errors worden gelogd.

11. `scripts/fetch\_pp\_scores.py`

Dit script vult `data/pp\_scores.json`.

11.1 Arguments

```
parser.add_argument("--accounts", type=Path, default=ACCOUNTS_PATH)
parser.add_argument("--output", type=Path, default=PP_SCORES_PATH)
parser.add_argument("--log-file", type=Path, default=Path("data/fetch_pp_scores.log"))
)
parser.add_argument("--batch-size", type=int, default=100)
parser.add_argument("--sleep-seconds", type=float, default=0.1)
```

Belangrijk: je gebruikte zelf vaak `--sleep-seconds 36` om binnen rate limits te blijven.

11.2 Foutafhandeling in `main`

```
try:
    return run(args)
except KeyboardInterrupt:
    log_message(args.log_file, "Stopped by user")
    return 130
except Exception:
    error_text = traceback.format_exc()
    log_message(args.log_file, error_text.rstrip())
    return 1
```

Als je het proces stopt met Ctrl+C, wordt dat gelogd. Bij echte errors komt de traceback in het logfile.

11.3 Accounts en cache

```
accounts = read_ids(args.accounts)
if args.limit_accounts is not None:
    accounts = accounts[: args.limit_accounts]
```

`--limit-accounts` is handig voor smoke tests.

```
pp_scores = {str(key): int(value) for key, value in load_json(args.output, {}).items()}
pending_accounts = [account for account in accounts if account not in pp_scores]
```

Alleen accounts die nog niet in de cache zitten worden opgehaald.

Dit is waarom je niet telkens opnieuw alle accounts opvraagt.

11.4 Batches

```
batches = chunked(pending_accounts, max(1, min(args.batch_size, 100)))
```

Zelfs als je per ongeluk `--batch-size 1000` meegeeft, wordt het maximaal 100.

11.5 Retry bij rate limit

```
except StatlockerRateLimitError as exc:
    wait_seconds = exc.retry after seconds or args.rate limit sleep seconds
    time.sleep(wait_seconds)
```

Bij rate limit stopt het script niet, maar wacht het.

11.6 Failed batch

```
except StatlockerApiError as exc:
    log message(args.log file, f"[batch ...] failed: {exc}")
    payload = []
    break
```

Een gewone API-error zorgt ervoor dat die batch wordt overgeslagen. Het script gaat daarna verder met de volgende batch.

11.7 Cache opslaan per batch

```
fetches = extract pp scores(payload)
pp_scores.update(fetches)
write_json(args.output, pp_scores)
```

Dit is bewust. Als het script halverwege stopt, zijn alle vorige batches al veilig opgeslagen.

12. `scripts/build\_dataset.py`

Dit script maakt de CSV waarmee het model traint.

12.1 Base columns

```
BASE_COLUMNS = [
    "match id",
    "start time s",
    "start time utc",
    "patch",
    "winner",
    "winner team index",
    "team 1 average badge",
    "team 2 average badge",
    "missing pp scores",
]
```

Niet alles hiervan wordt gebruikt door het neural network, maar het is nuttig voor:

- debugging;
- filtering;

- train/test split op tijd;
- datasetkwaliteit controleren.

12.2 Dynamische kolommen

```
*[f"team_1_hero_{index}" for index in range(1, 7)]
```

Maakt:

```
team 1 hero 1
team 1 hero 2
...
team_1_hero_6
```

Hetzelfde gebeurt voor team 2, accounts en ppScores.

12.3 Main flow

```
matches = read_jsonl(args.matches)
pp_scores = {str(key): str(value) for key, value in load_json(args.pp_scores, {}).items()}
rows = [build_row(match, pp_scores) for match in matches]
```

De inputbronnen:

- match JSONL;
- ppScore cache.

De output:

- lijst CSV-rijen.

12.4 Complete ppScore filter

```
if args.require_complete_pp:
    rows = [row for row in rows if row["missing_pp_scores"] == "0"]
```

Dit maakt een dataset waar elke match 12 bekende ppScores heeft. Die is schoner, maar kleiner.

12.5 CSV schrijven

```
with args.output.open("w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=FIELDNAMES)
    writer.writeheader()
    writer.writerows(rows)
```

`newline=""` is de juiste manier om CSV te schrijven in Python.

12.6 `build\_row`

```
row = {
    "match id": str(match.get("match id", "")),
    "start time s": value or empty(match.get("start time s")),
    ...
}
```

Hier worden match-level velden gevuld.

```
for team index in (1, 2):
    players = list(match.get(f"team_{team_index}_players", []) or [])[:6]
```

Neem maximaal 6 spelers per team.

```
for slot in range(1, 7):
    player = players[slot - 1] if slot <= len(players) else {}
```

Ook als een match minder spelers heeft, worden de kolommen toch aangemaakt.

```
account id = value or empty(player.get("account id"))
row[f"team {team index} hero {slot}"] = value or empty(player.get("hero id"))
row[f"team {team index} account {slot}"] = account id
row[f"team_{team_index}_pp_score_{slot}"] = pp_scores.get(account_id, "")
```

Dit koppelt Statlocker ppScore aan de juiste speler via `account\_id`.

13. `scripts/train\_neural\_teamcomp.py` - config

Dit is het belangrijkste ML-bestand.

13.1 Imports

```
import argparse
import csv
import json
import math
import random
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
```

Waarom:

- `argparse`: CLI hyperparameters.

- `csv`: dataset lezen.
- `json`: metadata schrijven.
- `random`, `numpy`, `torch`: seed zetten.
- `dataclass`: modelconfig netjes opslaan.
- `torch`, `nn`: neural network.
- `DataLoader`, `Dataset`: batching.

13.2 Defaults

```
DEFAULT_DATASET = Path("data/team comp dataset 2026-05-22.csv")
DEFAULT_MODEL = Path("models/2026-05-22/neural_teamcomp_heroes_only.pt")
```

De defaults zijn niet altijd je finale model. Bij trainen geef je meestal expliciet `--dataset` en `--model-output`.

13.3 `ModelConfig`

```
@dataclass
class ModelConfig:
    embedding dim: int
    hidden dim: int
    dropout: float
    architecture: str
    use badge: bool
    use pp score: bool
    extra feature dim: int
    num heroes: int
    activation: str = "relu"
```

Dit object beschrijft de architectuur. Het wordt opgeslagen in de `.pt` checkpoint en `.json` metadata. De Streamlit app gebruikt dit om hetzelfde model opnieuw op te bouwen.

14. `TeamCompDataset`

14.1 Klasse-definitie

```
class TeamCompDataset(Dataset[tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]]):
```

Een PyTorch `Dataset` moet minstens hebben:

- `\_\_len\_\_`
- `\_\_getitem\_\_`

14.2 Samples bouwen

```
self.samples: list[tuple[list[int], list[int], list[float], float]] = []
```

Elke sample bevat:

```
team 1 hero indices  
team 2 hero indices  
extra features  
target
```

14.3 Heroes naar indices

```
team 1 = [hero_to_index[row[f"team 1 hero {index}"]] for index in range(1, 7)]  
team_2 = [hero_to_index[row[f"team 2 hero {index}"]] for index in range(1, 7)]
```

De CSV bevat hero IDs zoals `"77"`. Het model wil indices van `0` tot `num\_heroes - 1`.

14.4 Extra features

```
extra = extra_features(row, use_badge=use_badge, use_pp_score=use_pp_score)
```

Afhankelijk van flags krijg je:

- niets;
- badgeverschil;
- ppScorefeatures;
- beide.

14.5 Target

```
target = 1.0 if row["winner_team_index"] == "1" else 0.0
```

Het model voorspelt team 2 win probability.

Dus:

```
winner team index == "1" -> team 2 won -> y = 1  
winner_team_index == "0" -> team 1 won -> y = 0
```

14.6 Swap augmentation

```
if augment swap:  
    swapped_extra = swap_extra_features(...)  
    self.samples.append((team_2, team_1, swapped_extra, 1.0 - target))
```

Als je teamvolgorde omdraait, moet het label ook omdraaien.

Voorbeeld:

```
Origineel: A vs B, B wint -> y = 1
Swapped:   B vs A, A wint -> y = 0
```

Dit verdubbelt trainingdata en maakt het model minder afhankelijk van arbitraire teamvolgorde.

14.7 Tensor output

```
return (
    torch.tensor(team 1, dtype=torch.long),
    torch.tensor(team 2, dtype=torch.long),
    torch.tensor(extra, dtype=torch.float32),
    torch.tensor([target], dtype=torch.float32),
)
```

Waarom `team\_1` en `team\_2` `long` zijn:

- Embedding layers verwachten integer indices.

Waarom `extra` en `target` floats zijn:

- Neural network berekeningen gebeuren met floats.

15. `TeamCompNet`

15.1 Embedding layer

```
self.hero_embedding = nn.Embedding(config.num_heroes, config.embedding_dim)
```

Wiskundig:

$$E \in \mathbb{R}^{(H \times d)}$$

waar:

- `H = num\_heroes`
- `d = embedding\_dim`

Voor hero index `i`:

$$x_i = E[i]$$

`E` zijn trainbare weights. Ook deze krijgen updates:

$$E' = E - \eta * \partial L / \partial E$$

15.2 Inputdimensie

```
input_dim = self._encoded_dim(config) + config.extra_feature_dim
```


Voor het gekozen model:

```
architecture = pool
embedding dim = 24
encoded dim = 9 * 24 = 216
extra feature dim = 6
input_dim = 222
```

15.3 MLP

```
self.network = nn.Sequential(
    nn.Linear(input dim, config.hidden dim),
    activation layer(config.activation),
    nn.Dropout(config.dropout),
    nn.Linear(config.hidden dim, config.hidden dim // 2),
    activation layer(config.activation),
    nn.Dropout(config.dropout),
    nn.Linear(config.hidden dim // 2, 1),
)
```

Voor `hidden\_dim=128`:

```
Linear 222 -> 128
SiLU
Dropout
Linear 128 -> 64
SiLU
Dropout
Linear 64 -> 1
```

De laatste output is een logit, nog geen probability.

15.4 Forward pass

```
team_1_embeddings = self.hero_embedding(team_1)
team_2_embeddings = self.hero_embedding(team_2)
```

Input shape:

```
team_1: batch_size x 6
```

Output shape:

```
team_1_embeddings: batch_size x 6 x embedding_dim
```

15.5 Mean pooling

```
team_1_mean = team_1_embeddings.mean(dim=1)
team_2_mean = team_2_embeddings.mean(dim=1)
```

Dit neemt het gemiddelde over de 6 heroes. Shape:

```
batch_size x embedding_dim
```

Mean pooling is volgorde-ongevoelig: het maakt niet uit of hero A slot 1 of slot 4 staat.

15.6 Feature concatenatie

```
features = self.encode_teams(...)
if extra.shape[1] > 0:
    features.append(extra)
return self.network(torch.cat(features, dim=1))
```

Alle featureblokken worden naast elkaar geplakt tot één vector `x`.

Daarna gaat `x` door de MLP.

16. Team encoding in detail

16.1 Basisfeatures

```
features = [
    team_1_mean,
    team_2_mean,
    team_2_mean - team_1_mean,
    torch.abs(team_2_mean - team_1_mean),
    team_1_mean * team_2_mean,
]
```

Uitleg:

1. `team\_1\_mean`: globale representatie team 1.
2. `team\_2\_mean`: globale representatie team 2.
3. `team\_2\_mean - team\_1\_mean`: richting van verschil.
4. `abs(...)`: grootte van verschil zonder richting.
5. `team\_1\_mean \* team\_2\_mean`: interactie tussen teams.

16.2 Max pooling

```
team_1_max = team_1_embeddings.max(dim=1).values
team_2_max = team_2_embeddings.max(dim=1).values
```

Max pooling pakt per embeddingdimensie de sterkste activatie binnen het team. Waarom nuttig? Soms is een specifieke hero-eigenschap aanwezig zodra één hero

die bezit. Mean pooling kan dat afzwakken, max pooling bewaart het sterker.

16.3 Pool architecture

```
features.extend([
    team 1 max,
    team 2 max,
    team 2 max - team 1 max,
    torch.abs(team 2 max - team 1 max),
])
```

Samen met de 5 meanfeatures zijn dit 9 embeddingblokken.

Dus:

```
encoded_dim = 9 * embedding_dim
```

16.4 Matchup architecture

```
if self.config.architecture == "matchup":
    features.append(pairwise_summary(team_1_embeddings, team_2_embeddings))
```

Deze architectuur voegt expliciete pairwise hero-interacties toe. In experimenten was de gekozen `pool` variant beter/handiger.

17. ppScorefeatures in detail

17.1 Team scores lezen

```
team 1 scores = [
    value
    for index in range(1, 7)
    if (value := int or none(row.get(f"team 1 pp score {index}"))) is not None
]
```

Dit:

- leest `team\_1\_pp\_score\_1` tot `team\_1\_pp\_score\_6`;
- zet waarden om naar int;
- slaat missende waarden over.

Hetzelfde gebeurt voor team 2.

17.2 Coverage

```
observed_scores = team_1_scores + team_2_scores
```

```
coverage = len(observed_scores) / 12.0
```

Als 9 van 12 spelers ppScore hebben:

```
coverage = 9 / 12 = 0.75
```

Dit vertelt het model hoe betrouwbaar de rankinformatie is.

17.3 Lobby mean

```
lobby mean = sum(observed scores) / len(observed scores) / 10000.0 if observed scores  
else 0.0
```

Voor een lobbygemiddelde van 4200:

```
lobby_mean = 4200 / 10000 = 0.42
```

Waarom delen door 10000? Neural networks trainen stabiel als features ongeveer in kleine ranges zitten.

17.4 Als een team geen ppScores heeft

```
if not team 1 scores or not team 2 scores:  
    return [0.0, 0.0, 0.0, 0.0, coverage, lobby_mean]
```

Als één team geen scores heeft, kan je geen eerlijk teamverschil berekenen. Coverage en lobby mean blijven wel nuttig.

17.5 Verschilfeatures

```
team 1 mean = sum(team 1 scores) / len(team 1 scores)  
team 2 mean = sum(team 2 scores) / len(team 2 scores)  
return [  
    (team 2 mean - team 1 mean) / 10000.0,  
    (min(team 2 scores) - min(team 1 scores)) / 10000.0,  
    (max(team 2 scores) - max(team 1 scores)) / 10000.0,  
    (len(team 2 scores) - len(team 1 scores)) / 6.0,  
    coverage,  
    lobby mean,  
]
```

Feature 1:

```
gemiddelde sterkte team 2 - gemiddelde sterkte team 1
```

Feature 2:

```
laagste speler team 2 - laagste speler team 1
```


Feature 3:

```
hoogste speler team 2 - hoogste speler team 1
```

Feature 4:

```
aantal bekende scores team 2 - aantal bekende scores team 1
```

Feature 5:

```
algemene dekking van ppScores
```

Feature 6:

```
absoluut lobbyniveau
```

18. Wiskunde van het neural network

18.1 Input

Na embedding en pooling krijgt het MLP een vector:

```
 $x \in \mathbb{R}^{222}$ 
```

voor het geselecteerde ppScore-context model.

18.2 Eerste lineaire laag

```
 $z_1 = w_1 x + b_1$ 
```

Shapes:

```
 $x \in \mathbb{R}^{222}$   
 $w_1 \in \mathbb{R}^{(128 \times 222)}$   
 $b_1 \in \mathbb{R}^{128}$   
 $z_1 \in \mathbb{R}^{128}$ 
```

Elke neuron heeft eigen weights en bias.

18.3 Activatie

Het gekozen model gebruikt SiLU:

```
 $a_1 = \text{SiLU}(z_1)$   
 $\text{SiLU}(z) = z * \text{sigmoid}(z)$   
 $\text{sigmoid}(z) = 1 / (1 + \exp(-z))$ 
```

Waarom activatie? Zonder nonlineariteit zou meerdere lineaire lagen samen nog

altijd één lineaire functie zijn.

18.4 Tweede laag

```
z_2 = w_2 a_1 + b_2  
a_2 = SiLU(z_2)
```

Shapes:

```
w_2 ∈ R^(64 x 128)  
b_2 ∈ R^64  
z_2 ∈ R^64  
a_2 ∈ R^64
```

18.5 Outputlaag

```
z_out = w_3 a_2 + b_3
```

Shapes:

```
w_3 ∈ R^(1 x 64)  
b_3 ∈ R^1  
z_out ∈ R^1
```

Daarna:

```
p = sigmoid(z_out)
```

`p` is de kans dat team 2 wint.

18.6 Loss

Binary cross entropy:

```
L = -[ y log(p) + (1 - y) log(1 - p) ]
```

Voorbeelden:

```
y = 1, p = 0.90 -> lage loss  
y = 1, p = 0.10 -> hoge loss  
y = 0, p = 0.10 -> lage loss  
y = 0, p = 0.90 -> hoge loss
```

18.7 Updates

Basis gradient descent:

```
w' = w - η * ∂L/∂w
```

$$b' = b - \eta * \partial L / \partial b$$

Voor laag 1:

$$\begin{aligned} w_{1'} &= w_1 - \eta * \partial L / \partial w_1 \\ b_{1'} &= b_1 - \eta * \partial L / \partial b_1 \end{aligned}$$

Voor laag 2:

$$\begin{aligned} w_{2'} &= w_2 - \eta * \partial L / \partial w_2 \\ b_{2'} &= b_2 - \eta * \partial L / \partial b_2 \end{aligned}$$

Voor outputlaag:

$$\begin{aligned} w_{3'} &= w_3 - \eta * \partial L / \partial w_3 \\ b_{3'} &= b_3 - \eta * \partial L / \partial b_3 \end{aligned}$$

Voor embeddings:

$$E' = E - \eta * \partial L / \partial E$$

In code gebeurt dit door:

```
loss.backward()  
optimizer.step()
```

18.8 AdamW

De code gebruikt:

```
optimizer = torch.optim.AdamW(model.parameters(), lr=args.learning_rate, weight_decay=args.weight_decay)
```

AdamW is geen simpele gradient descent, maar het idee blijft:

parameters bewegen in de richting die L verlaagt

AdamW houdt per parameter bewegende gemiddelden bij van gradients. Daardoor kan de effectieve stapgrootte per weight verschillen.

19. Training loop in detail

19.1 Model en optimizer

```
model = TeamCompNet(config).to(device)  
optimizer = torch.optim.AdamW(...)  
criterion = nn.BCEWithLogitsLoss()
```

`device` is `cuda` als GPU beschikbaar is, anders CPU.

`BCEWithLogitsLoss` verwacht logits, niet probabilities. Daarom zit `sigmoid` niet in de modeloutput tijdens training.

19.2 DataLoaders

```
train_loader = DataLoader(train_dataset, batch_size=args.batch_size, shuffle=True)
validation_loader = DataLoader(validation_dataset, batch_size=args.batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=args.batch_size, shuffle=False)
```

Training wordt geshuffled. Validation en test niet, want daar wil je alleen meten.

19.3 Epoch loop

```
for epoch in range(1, args.epochs + 1):
    model.train()
```

`model.train()` zet dropout aan.

19.4 Batch loop

```
for team 1, team 2, extra, target in train_loader:
    team 1 = team 1.to(device)
    team 2 = team 2.to(device)
    extra = extra.to(device)
    target = target.to(device)
```

Alle tensors moeten op hetzelfde device als het model staan.

19.5 Forward en loss

```
optimizer.zero_grad()
logits = model(team 1, team 2, extra)
loss = criterion(logits, target)
```

`logits` zijn ruwe outputs. De lossfunctie past intern de juiste sigmoid/log-loss combinatie toe.

19.6 L1 toevoegen

```
if args.l1_lambda > 0:
    loss = loss + args.l1_lambda * l1_penalty(model)
```

Wiskundig:

$$L_{\text{total}} = L_{\text{BCE}} + \lambda * \sum |w|$$

19.7 Backprop en update

```
loss.backward()
if args.grad_clip > 0:
    nn.utils.clip_grad_norm (model.parameters(), args.grad_clip)
optimizer.step()
```

Dit is het echte leren.

19.8 Evaluatie na elke epoch

```
train_metrics = evaluate(model, train_loader, device)
validation_metrics = evaluate(model, validation_loader, device)
test_metrics = evaluate(model, test_loader, device)
```

De code print test metrics elke epoch. Voor wetenschappelijke strengheid kies je modellen vooral op validation, niet op test.

19.9 Best state bewaren

```
if validation_metrics["log loss"] < best_validation_log_loss - args.min_delta:
    best_validation_log_loss = validation_metrics["log loss"]
    best_epoch = epoch
    epochs_without_improvement = 0
    best_state = {name: value.detach().cpu().clone() for name, value in model.state_dict().items()}
```

De beste weights worden gekopieerd. Zonder clone zouden ze verder aangepast kunnen worden.

19.10 Early stopping

```
else:
    epochs_without_improvement += 1
    if epochs_without_improvement >= args.patience:
        break
```

Als validation loss te lang niet verbetert, stopt training. Dit helpt tegen overfitting.

19.11 Save

```
torch.save({...}, args.model_output)
```

De `pt` bevat alles wat nodig is om het model opnieuw te laden.

```
metadata_path.write_text(json.dumps(..., indent=2, sort_keys=True))
```

De ``.json`` is leesbaar voor mensen en handig om experimenten te vergelijken.

20. Evaluatie in detail

20.1 No grad

```
@torch.no_grad()  
def evaluate(...):
```

Tijdens evaluatie hoeven geen gradients berekend te worden. Dat bespaart geheugen en tijd.

20.2 Eval mode

```
model.eval()
```

Dropout gaat uit. Evaluatie moet deterministisch zijn.

20.3 Probability

```
probabilities = torch.sigmoid(logits)  
predictions = (probabilities >= 0.5).float()
```

Als probability minstens 0.5 is, voorspelt het model team 2.

20.4 Metrics

```
return {  
    "accuracy": correct / total,  
    "log loss": total_loss / total,  
    "positive rate": positives / total,  
    "avg predicted team2 win probability": probability_sum / total,  
}
```

``positive\_rate`` vertelt hoeveel echte team-2-wins er zijn. Dat helpt om te checken of de dataset ongeveer gebalanceerd is.

21. Experiment runners

21.1 ``scripts/run\_neural\_experiments.py``

Dit script test heroes-only modellen.

Het bevat een lijst configs:

```
EXPERIMENTS = [
```

```

{
    "name": "pool regularized",
    "epochs": "80",
    "embedding dim": "16",
    "hidden dim": "96",
    "dropout": "0.40",
    "architecture": "pool",
    "learning rate": "0.0005",
    "weight decay": "0.002",
    "l1 lambda": "0.0000005",
},
...
]

```

Per experiment bouwt het een command:

```

command = [
    sys.executable,
    str(script),
    "--dataset",
    str(args.dataset),
    "--model-output",
    str(model_path),
    ...
]

```

Daarna:

```
result = subprocess.run(command, check=False)
```

Het script start dus het trainingsscript als subprocess.

Na training leest het metadata:

```
metadata = json.loads(model_path.with_suffix(".json").read_text(encoding="utf-8"))
```

en schrijft een summary CSV.

21.2 `scripts/run\_ppscore\_context\_experiments.py`

Dit doet hetzelfde, maar met `--use-pp-score`.

Belangrijk:

```

command = [
    ...
    "--architecture",
    "pool",
    "--activation",
    experiment["activation"],
    ...
    "--use-pp-score",

```

```
] ]
```

Hier werden varianten getest met:

- `gelu`;
- `silu`;
- andere hidden sizes;
- andere learning rates;
- andere regularization.

Het geselecteerde model gebruikt rank-context omdat de app moet reageren op absolute lobby rank.

22. `scripts/download\_hero\_assets.py`

Dit script haalt hero-afbeeldingen op.

22.1 Defaults

```
DEFAULT HEROES URL = "https://api.deadlock-api.com/v1/assets/heroes"  
DEFAULT OUTPUT DIR = Path("app/assets/heroes")  
DEFAULT_MANIFEST = Path("app/assets/hero_images.json")
```

Het script downloadt afbeeldingen naar `app/assets/heroes` en schrijft een manifest.

22.2 Heroes ophalen

```
heroes = fetch_json(args.heroes_url)
```

`fetch\_json` verwacht een lijst van hero dicts.

22.3 Filter op hero IDs

```
allowed_ids = set(args.hero_ids or [])  
...  
if not hero_id or allowed_ids and hero_id not in allowed_ids:  
    continue
```

Als je `--hero-ids` meegeeft, downloadt het alleen die heroes.

22.4 Image URL kiezen

```
images = hero.get("images") or {}  
image_url = images.get(args.image_kind) or images.get(args.fallback_image_kind)
```

Eerst probeert het `icon\_hero\_card\_webp`, anders fallback `icon\_image\_small\_webp`.

22.5 Manifest

```
manifest[hero id] = {
    "name": str(hero.get("name") or f"Hero {hero id}"),
    "image": str(image path.relative to(args.manifest.parent)),
    "source": image url,
}
```

De app gebruikt dit manifest om te weten welk bestand bij welke hero ID hoort.

23. `app/streamlit\_app.py`

Dit is de user interface.

23.1 Imports en importpad

```
REPO_ROOT = Path( file ).resolve().parents[1]
TRAINING_DIR = REPO_ROOT / "scripts"
if str(TRAINING_DIR) not in sys.path:
    sys.path.insert(0, str(TRAINING_DIR))
```

De app importeert uit `scripts/train\_neural\_teamcomp.py`:

```
from train_neural_teamcomp import ModelConfig, TeamCompNet, pp_score_features
```

Dat is belangrijk: de app gebruikt dezelfde modelklasse en dezelfde ppScorefeaturefunctie als training.

23.2 Modelpad

```
MODEL_PATH = REPO_ROOT / "models" / "2026-05-22" / "neural_teamcomp_heroes_ppscore_context.pt"
METADATA_PATH = MODEL_PATH.with_suffix(".json")
```

Dit is het model dat de app gebruikt.

Als je een nieuw best model hebt, moet je dit pad aanpassen of het nieuwe model naar deze naam kopiëren.

23.3 Hero names

```
HERO_NAMES = {
    "1": "Infernus",
    "2": "Seven",
    ...
}
```

De API gebruikt IDs. De UI toont namen.

23.4 Model laden

```
@st.cache resource
def load_model() -> tuple[TeamCompNet, dict[str, int], dict]:
    checkpoint = torch.load(MODEL_PATH, map_location="cpu", weights_only=False)
    config = ModelConfig(**checkpoint["config"])
    model = TeamCompNet(config)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()
    metadata = json.loads(METADATA_PATH.read_text(encoding="utf-8"))
    return model, checkpoint["hero_to_index"], metadata
```

Regel per regel:

- `@st.cache_resource`: laad het model niet opnieuw bij elke UI-interactie.
- `torch.load`: lees checkpoint.
- `ModelConfig(**checkpoint["config"])`: bouw configobject.
- `TeamCompNet(config)`: maak modelarchitectuur.
- `load_state_dict`: vul getrainde weights.
- `eval()`: zet dropout uit.
- metadata lezen voor sidebar.

23.5 Hero images laden

```
@st.cache data
def load_hero_images() -> dict[str, Path]:
```

`@st.cache_data` is geschikt voor gewone data, zoals manifestbestanden.

```
if not HERO_IMAGE_MANIFEST_PATH.exists():
    return {}
```

Als afbeeldingen ontbreken, werkt de app nog steeds met fallbacks.

23.6 Ranklabel

```
def pp_score_rank_label(pp_score: int) -> str:
    if pp_score >= 6600:
        return "Eternus 6+"
    if pp_score >= 6000:
        return f"Eternus {(pp_score - 6000) // 100 + 1}"
```

Boven 6600 is Statlocker uncapped/Eternus 6+.

```
tier_index = min(pp_score // 600, len(RANK_TIERS) - 1)
sub_rank = (pp_score % 600) // 100 + 1
```

Elke tier is 600 PP en elke subrank 100 PP.

23.7 Prediction function

```
team 1 tensor = torch.tensor([[hero to index[hero id] for hero id in team 1]], dtype=
torch.long)
team 2 tensor = torch.tensor([[hero to index[hero id] for hero id in team 2]], dtype=
torch.long)
```

De dubbele brackets maken batch size 1:

```
shape = 1 x 6
```

```
score row = {
    **{"team 1 pp score {index}": str(score) for index, score in enumerate(team 1 sc
ores, start=1)},
    **{"team 2 pp score {index}": str(score) for index, score in enumerate(team 2 sc
ores, start=1)},
}
```

De app bouwt een mini-CSV-rij zodat `pp\_score\_features` dezelfde inputvorm krijgt als tijdens training.

```
extra_tensor = torch.tensor([pp_score_features(score_row)], dtype=torch.float32)
```

Ook hier batch size 1.

```
with torch.no_grad():
    logits = model(team 1 tensor, team 2 tensor, extra tensor)
    return float(torch.sigmoid(logits).item())
```

Geen gradients nodig voor inference.

23.8 Lineup picker

```
columns = st.columns(6)
for index in range(6):
    with columns[index]:
        render hero portrait(...)
        pick = st.selectbox(...)
```

Elke teamrij heeft 6 kolommen: één per hero.

```
if score mode == "Per player":
    score = st.number input(...)
    scores.append(score)
else:
    scores.append(int(team_score or DEFAULT_PP_SCORE))
```

De app ondersteunt:

- één gemiddelde teamrank;
- individuele ppScores per speler.

23.9 Randomize

```
if st.button("Randomize", use_container_width=True):
    randomized = random.sample(options, 12)
```

`random.sample` kiest 12 unieke heroes.

Daarna worden 6 naar team 1 en 6 naar team 2 geschreven in `st.session\_state`.

23.10 Duplicate warning

```
all_picks = team 1 + team 2
duplicate_ids = sorted({hero_id for hero_id in all_picks if all_picks.count(hero_id) > 1}, key=int)
if duplicate_ids:
    st.warning(...)
```

Deadlock drafts kunnen geen duplicate heroes hebben. De app blokkeert het niet hard, maar waarschuwt de gebruiker.

23.11 Resultaat tonen

```
team_2_probability = predict_team_2_probability(...)
team_1_probability = 1.0 - team_2_probability
```

Omdat het model team 2 probability geeft, is team 1 probability het complement.

```
result_left.metric("Team 1 win chance", f"{team_1_probability * 100:.2f}%")
result_mid.metric("Team 2 win chance", f"{team_2_probability * 100:.2f}%")
result_right.progress(team_2_probability, text="Team 2 probability")
```

De app toont zowel cijfers als een progress bar.

24. Volledige dataflow met bestanden

24.1 Matchdata

Input:

```
Deadlock API
```

Command:

```
.venv/bin/python scripts/fetch_matches.py \
    --output data/matches_2026-05-22.jsonl \
```



```
--state-file data/fetch state 2026-05-22.json \  
--target-count 20000 \  
--batch-size 100 \  
--game-mode normal \  
--required-patch 2026-05-22 \  
--order-direction desc \  
--sleep-seconds 6.2
```

Output:

```
data/matches 2026-05-22.jsonl  
data/fetch_state_2026-05-22.json
```

24.2 Account IDs

Command:

```
.venv/bin/python scripts/extract accounts.py \  
--matches data/matches 2026-05-22.jsonl \  
--output data/accounts_2026-05-22_20k.txt
```

Output:

```
data/accounts_2026-05-22_20k.txt
```

24.3 ppScores

Command:

```
.venv/bin/python scripts/fetch pp scores.py \  
--accounts data/accounts 2026-05-22 20k.txt \  
--output data/pp scores.json \  
--log-file data/fetch pp scores 2026-05-22.log \  
--batch-size 100 \  
--sleep-seconds 36
```

Output:

```
data/pp scores.json  
data/fetch_pp_scores_2026-05-22.log
```

24.4 Dataset

Command:

```
.venv/bin/python scripts/build dataset.py \  
--matches data/matches 2026-05-22.jsonl \  
--pp-scores data/pp scores.json \  
--output data/team_comp_dataset_2026-05-22_20k.csv
```

Output:

```
data/team_comp_dataset_2026-05-22_20k.csv
```

24.5 Training

Command:

```
.venv/bin/python scripts/train_neural_teamcomp.py \  
  --dataset data/team_comp_dataset_2026-05-22_20k.csv \  
  --model-output models/2026-05-22/neural_teamcomp_heroes_ppscore_context.pt \  
  --epochs 80 \  
  --architecture pool \  
  --activation silu \  
  --embedding-dim 24 \  
  --hidden-dim 128 \  
  --dropout 0.40 \  
  --learning-rate 0.0004 \  
  --weight-decay 0.0015 \  
  --l1-lambda 0.0000003 \  
  --patience 8 \  
  --batch-size 256 \  
  --use-pp-score
```

Output:

```
models/2026-05-22/neural_teamcomp_heroes_ppscore_context.pt  
models/2026-05-22/neural_teamcomp_heroes_ppscore_context.json
```

24.6 App

Command:

```
.venv/bin/streamlit run app/streamlit_app.py \  
  --server.address 0.0.0.0 \  
  --server.port 8501
```

Output:

```
Webapp op poort 8501
```

25. Hoe je dit verdedigt

Als gevraagd wordt "waar zit de AI?", wijs je naar:

```
scripts/train_neural_teamcomp.py
```

en leg je uit:

- ``TeamCompDataset`` maakt tensors;
- ``TeamCompNet`` is het neural network;
- ``nn.Embedding`` leert herorepresentaties;
- ``nn.Linear`` lagen vormen de MLP;
- ``BCEWithLogitsLoss`` traint binary classification;
- ``AdamW`` update weights;
- validation log loss kiest het beste model.

Als gevraagd wordt "waar zit de UI?", wijs je naar:

```
app/streamlit_app.py
```

Als gevraagd wordt "hoe voeg je nieuwe data toe?", geef je pipeline:

```
fetch matches -> extract accounts -> fetch pp scores -> build dataset -> train neural
_teamcomp
```

Als gevraagd wordt "waarom is dit deep learning?", antwoord je:

```
Het model is een PyTorch neural network met trainbare embeddings, meerdere lineaire l
agen,
nonlineaire activaties, dropout, backpropagation en gradient-based optimization.
```

26. Belangrijkste valkuilen

26.1 Hero ID is geen numerieke feature

Hero ID 77 betekent niet "meer" dan hero ID 1. Daarom gebruikt het model embeddings.

26.2 Test accuracy is niet alles

Een model kan hogere test accuracy hebben door toeval. Validation log loss is beter om modelkeuze te sturen.

26.3 ppScore partial data

Niet elke speler heeft ppScore. Daarom zijn ``coverage`` en ``missing_pp_scores`` belangrijk.

26.4 Patchdata mengen is riskant

Na een patch kunnen heroes anders werken. Daarom is ``required_patch`` belangrijk.

26.5 Street Brawl mag niet mee

Daarom staat ``--game-mode normal`` in fetch commands.

27. Eén-zin samenvatting

Dit project verzamelt patch-specifieke normale Deadlock matches, verrijkt ze met Statlocker ppScores, bouwt daaruit een trainbare dataset, traint een PyTorch neural network met hero embeddings en rankfeatures, en gebruikt dat model in een Streamlit webapp om zelf gekozen teamcomps te evalueren.